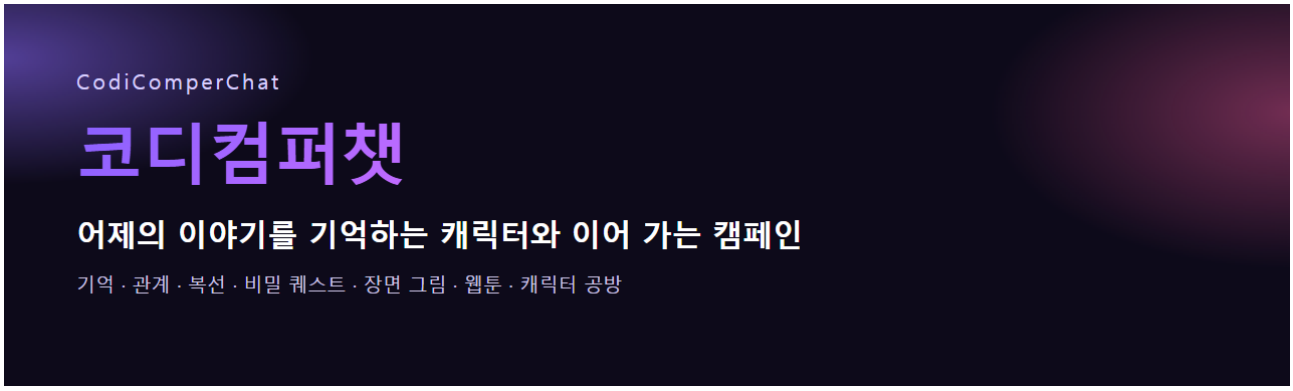


CodiComperChat Portfolio

어제의 이야기를 기억하는 캐릭터와 이어 가는 캠페인

이 사이트는 1인 풀스택 프로젝트「코디컴퍼챗」을 프론트엔드 · 운영 콘솔 · 백엔드 · AI 서비스 네 층으로 나눠 소개하는 포트폴리오입니다. 각 층이 무엇을 맡고, 어떤 판단으로 만들었고, 어디서 막혔으며, 앞으로 무엇을 할지를 숫자와 화면으로 보여 드립니다.

기간	2026-09-15 ~ 2026-11-07 (약 8주)
역할	1인 개발 (기획 · 프론트 · 백엔드 · AI · 배포)
서비스	https://codicomperchat-web.vercel.app
저장소	https://github.com/samam0234/CodiComperChat

규모	값
커밋	381
화면 (이용자 35 · 콘솔 12)	47
DB 테이블	37
Nest API 경로	155
API 시험	232
화면 E2E	123

목차

01 프론트엔드 — 이용자 웹 · Next.js 16

02 어드민 콘솔 — 운영자 웹 · Next.js 16

03 백엔드 서버 — 신뢰 경계 · Nest.js 12

04 AI 서비스 — 연기 · 판정 · 그림 · FastAPI

05 미래 지향 — 남은 일과 다음 단계

06 기획 스프레드시트 — 기획 · 요구사항 · 화면 · 일정

07 ERD — 테이블 37 · 외래 키 40

08 AI 가이드 챗봇 — LangGraph · RAG

프로젝트 소개

유저가 주인공인 이야기를, 성격이 흔들리지 않는 AI 캐릭터가 함께 이어 가는 웹 서비스. 대화를 닫았다 다시 열어도 캐릭터가 지난 일을 기억하고, 관계가 쌓이고, 숨겨 둔 비밀과 부탁이 때가 되면 드러난다.

반복되던 문제

- 대화가 길어지면 앞일을 잊는다
- 물어보면 설정에 없는 개인사를 지어낸다
- 비밀 설정을 유도하면 털어놓는다
- 대화 말고는 할 게 적다

차별점을 모델이 아니라 '상태'에 두었다. 무엇을 기억하고, 무엇을 숨기고, 무엇을 코드가 대신 판단할지를 설계했다.

보통의 캐릭터 채팅	코디컴퍼챗
긴 대화가 쌓이면 앞일을 잊는다	기억 · 요약 · 친밀도 · 복선을 따로 저장하고 필요한 만큼만 넣는다
없는 개인사를 지어낸다	설정에 근거가 없으면 단정하지 않게 막고, 지어냈으면 다시 쓴다
비밀을 유도하면 털어놓는다	비밀을 모델에 아예 보내지 않는다
대화 말고는 할 게 적다	속마음 · 일기 · 문자 · 게시판 · 장면 그림 · 웹툰으로 넓어진다
캐릭터는 프롬프트 한 덩어리	8단계 카드와 3턴 테스트를 통과해야 발행된다
한 번에 한 캐릭터와만	월드코퍼챗: 한 세계관의 여러 인물과 파트 · 지도 · 미니게임으로

간단한 기능 소개

- 이어지고 쌓이는 대화 — 장기 기억 · 지난 이야기 요약 · 친밀도 · 복선을 따로 저장. 유저가 잘못 잡힌 기억을 직접 지운다.
- 비밀 퀘스트 — 친밀도 문턱을 넘으면 캐릭터가 부탁을 꺼낸다. 달성은 모델이 아니라 코드가 인용을 대조해 판정한다.
- 월드코퍼챗 — 세계관 · 인물 · 장소 · 파트 · 지도로 여러 인물과 이야기를 잇는다. 미니게임 결과는 서버만 정한다.
- 캐릭터 공방 — 8단계 카드 + AI 초안 + 발행 전 3턴 테스트(말투 붕괴 · 비밀 누설 판정).
- 코디포토넷 · 웹툰 — 줄거리를 읽고 장면을 설계해 그린다. 모든 그림은 검수를 지나야 돌아오고, 막히면 코레딧 환불.
- 운영과 보안 — 운영 콘솔, 요청 한도, nonce CSP, 성인 수위 분리, 운영 환경 안전 검사(HTTPS 강제).

기술 스택

이용자 웹	Next.js 16 · React 19 · TypeScript · CSS Modules
운영 콘솔	Next.js 16 (웹과 쿠키 · 배포 분리)
백엔드	Nest.js 12 (ESM) · Prisma 7 · PostgreSQL 16
AI 서버	FastAPI · Python 3.12 · Ollama · diffusers
대화 모델	NVIDIA 호스팅 gemma-4-31b-it → 예비 Ollama gemma4:e4b

시험	Vitest · Playwright · unittest · 자체 평가 스크립트
인프라	Docker Compose · Caddy · GitHub Actions · Vercel · Oracle Cloud



시스템 구조

01 · FRONTEND

프론트엔드 — 이용자 웹

화면 35개의 Next.js 16 이용자 사이트. 판단은 하지 않고, 읽기는 서버 컴포넌트, 쓰기는 서버 액션으로 Nest만 부른다. 브라우저는 API 주소도 토큰도 모른다.

01. 개요

이용자가 캐릭터를 둘러보고, 대화하고, 공방에서 캐릭터를 만들고, 월드코퍼챗을 플레이하는 웹이다. 화면은 35개, 묶음은 둘러보기 · 대화 · 오퍼레이션 · 공방 · 월드 · 내 것 · 계정 · 요금 · 안내 아홉 갈래다.

- 서버 컴포넌트가 lib/*.ts(server-only)로 읽고, 클라이언트는 app/actions/*.ts 서버 액션으로 쓴다
- 로그인 토큰은 httpOnly 쿠키에만 있고 서버 코드가 Authorization 헤더로 붙인다 → CORS를 열지 않는다
- 운영 배포: Vercel(icn1), API는 Oracle VM의 Caddy 뒤

02. 사용 엔진

프레임워크	Next.js 16 (App Router · 서버 컴포넌트 · 서버 액션)
UI	React 19 · TypeScript · CSS Modules
스타일	디자인 토큰(app/globals.css) — 라이트 · 다크, 한국 시각에 따라 자동 전환
보안	proxy.ts가 요청마다 nonce CSP 발급, 외부 스크립트는 결제창 하나뿐
시험	Playwright E2E 123개(운영 빌드 + 가짜 LLM · 가짜 그림 엔진)

03. 상세 기능

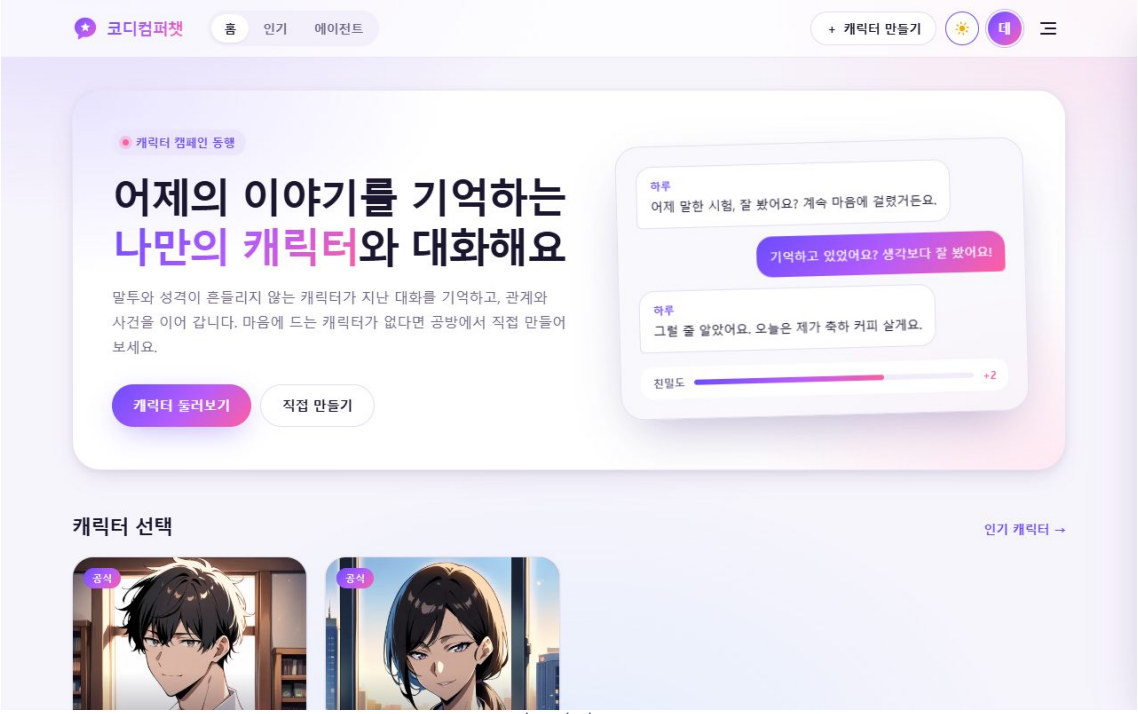
- 대화: 기억 · 지난 이야기 · 상태 설정 · 엑스트라 · 모델 고르기 · /다시 /계속 /되돌리기 명령어 · *행동* 과 @: 나레이션 꾸미기 · 대답 추천
- 오퍼레이션: 심리 탐구(속마음) · 일기 · 문자 · 가상 게시판 · 웹툰
- 공방: 8단계 카드 편집기, 칸마다 AI 초안, 3턴 테스트, 초상(파일 · AI)
- 월드코퍼챗: 세계관 편집기(인물 · 장소 · 파트 · 지도), 대화 방식 4가지(채팅 · 비주얼노벨 · 소설 · 메신저), 미니게임 창
- 글은 HTML로 넣지 않는다: lib/chat-format.ts가 표기를 읽고 React 요소로만 그린다(XSS 차단)

04. API 연결

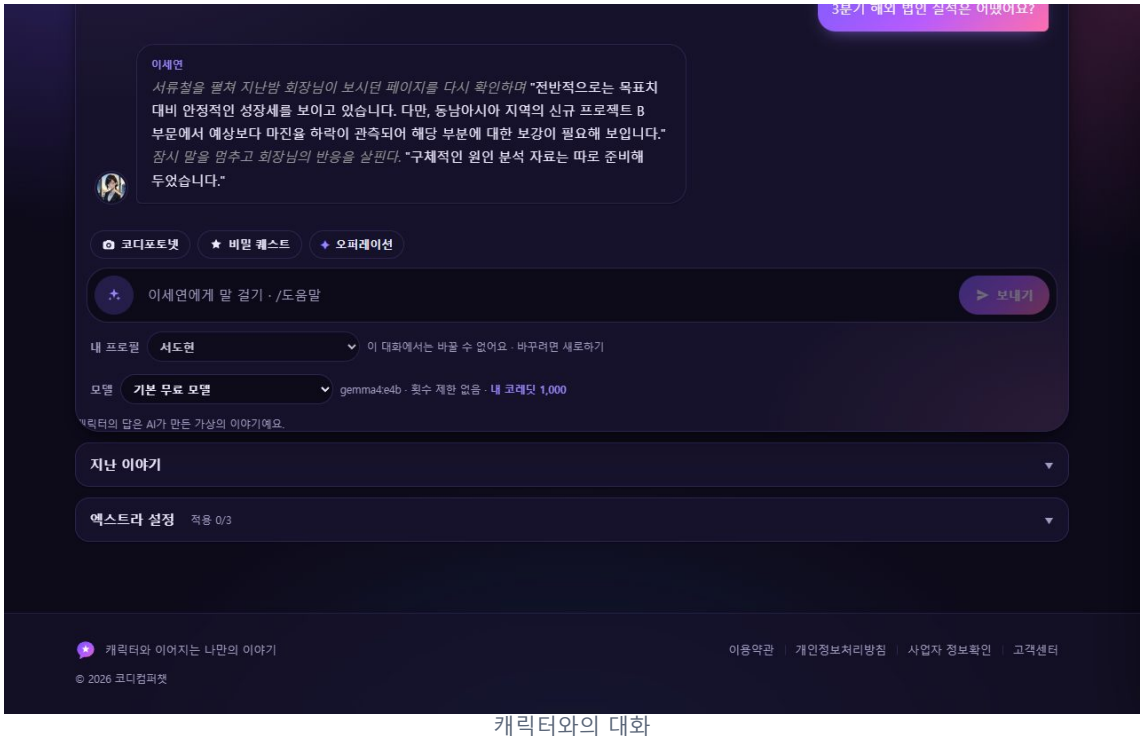
브라우저 → Next 서버 → Nest 순서만 허용한다. 브라우저에서 직접 부르는 길은 app/api/* 중계 라우트뿐이고, 여기에는 무거운 추론을 넣지 않는다.

읽기	서버 컴포넌트 → lib/*.ts → GET Nest (Bearer)
쓰기	클라이언트 → app/actions/*.ts(서버 액션) → POST Nest
대화 대기	app/api/characters/[id]/chat — 답을 기다리는 동안 대기 표시를 그리려고 쿠키 토큰을 붙여 넘긴다
그림	app/api/.../portrait · photonet · webtoon — <img>는 토큰을 못 붙여 서버가 받아 no-store로 돌려준다
이용자 IP	lib/client-ip.ts가 X-Forwarded-For + 프록시 토큰으로 넘긴다 → 요청 한도가 사람별로 센다

05. 내부 페이지



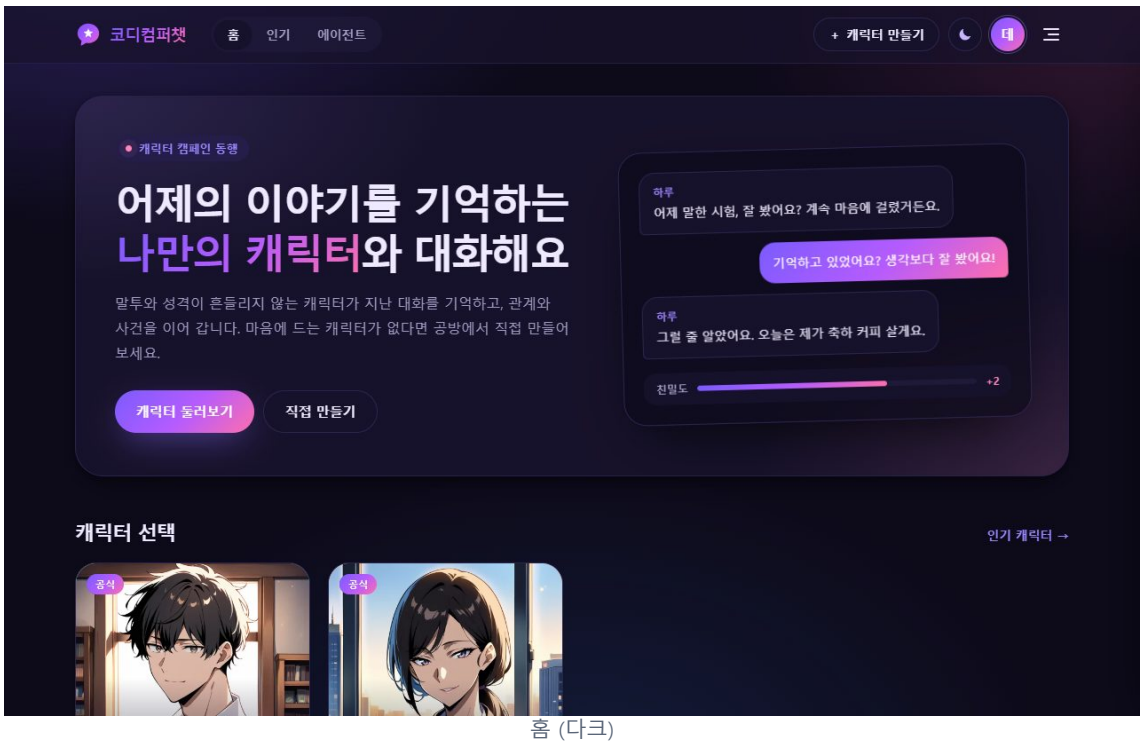
홈 (라이트)



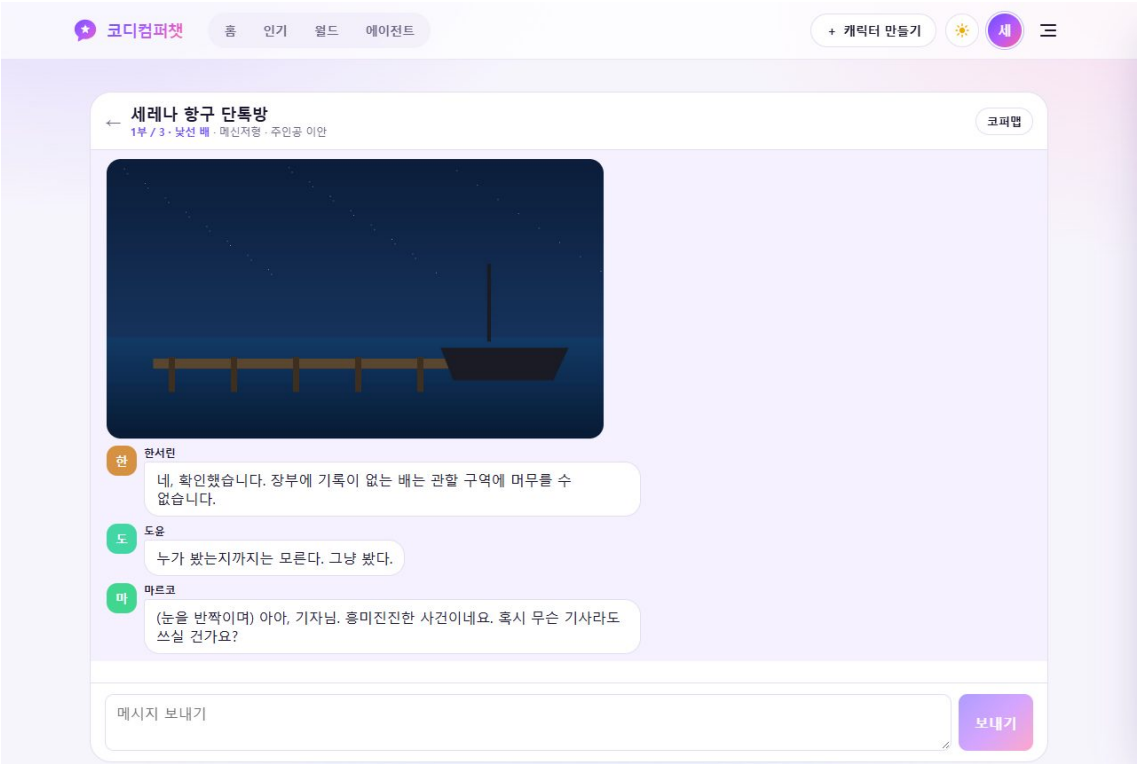
캐릭터와의 대화

06. 디자인

따뜻한 종이 · 구리 톤의 토큰 위에 라이트 · 다크 두 벌을 유지한다. 색 · 그림자 · 둥글기는 토큰만 쓰도록 규칙을 정해 테마가 깨지지 않게 했다. 대화는 말풍선 · 나레이션 · 행동 표기를 시각적으로 구분하고, 월드는 대화 방식마다 전혀 다른 레이아웃(무대 + 대사 상자, 문단, 메신저 말풍선)을 쓴다.



홈 (다크)



메신저형 월드

07. 문제점과 개선

좁은 화면(390px)	대화의 모델 고르기 칸이 눌려 글자가 안 보인다 → 고칠 예정(TASKS Q07)
응답 대기	스트리밍 응답이 없어 호스팅 API 한 번(약 19초)도 길게 느껴진다 → 스트리밍 도입 검토
하이드레이션	toLocaleString 불일치 → lib/format-time.ts로 날짜 표기 통일(해결)
e2e 무작위 실패	next dev의 메모리 압박 → 운영 빌드로 시험(해결)

08. 미래 지향

- 토큰 단위 스트리밍 응답(SSE)으로 체감 대기 단축
- 접근성 감사(키보드 · 스크린리더) 자동화
- PWA로 모바일 홈 화면 설치

02 · ADMIN CONSOLE

어드민 콘솔 — 운영자 웹

운영자 전용 Next.js 16 앱(화면 12개). 이용자 웹과 쿠키 · 배포를 나눠, 이용자 사이트에 문제가 생겨도 운영 도구가 같이 흔들리지 않게 했다.

01. 개요

신고 처리, 초상 · 그림 검수, 계정 정지와 등급 변경, 세계관 내리기와 이벤트 기록, 운영 설정을 한 곳에서 한다. 권한은 세 겹으로 확인한다.

- proxy.ts: 쿠키가 없으면 로그인으로
- 각 페이지의 requireAdmin(): Nest에 토큰을 확인, 역할 ADMIN
- Nest AdminGuard: DB의 역할을 다시 확인
- 공개 호스트는 CONSOLE_IP_ALLOWLIST에 있는 IP만 연다

02. 사용 엔진

프레임워크	Next.js 16 (웹과 같은 판, 별도 앱)
인증	콘솔 전용 쿠키, 운영자 계정만 로그인
보안	nonce CSP, X-Robots-Tag: noindex, IP 허용 목록
배포	Vercel 프로젝트 codicomperchat-console (API 연결은 허용 IP 결정 대기)

03. 상세 기능

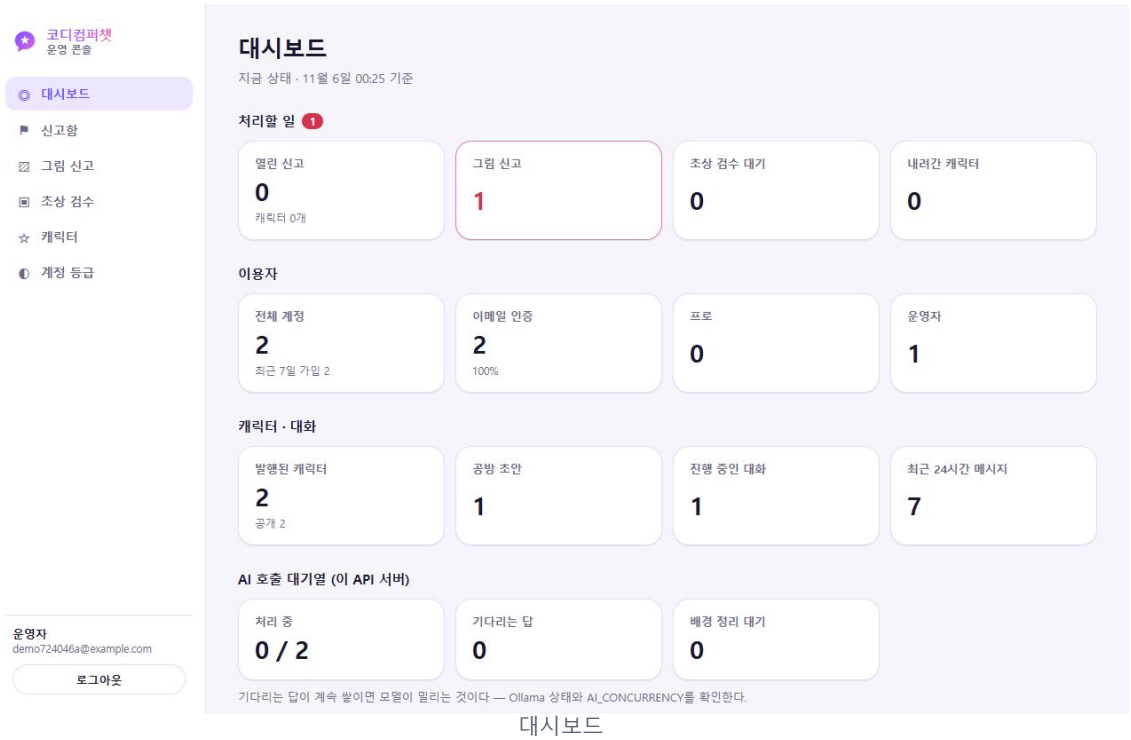
/	대시보드: 처리할 일 · 이용자 · 캐릭터 · AI 대기열
/reports	신고함: 내리기(사유 필수) · 기각 · 복구
/portraits	초상 검수: 자동 검수가 넘긴 것 승인 · 반려
/content	그림 신고: 접근 차단 · 계정 정지 · 해제
/characters	비공개 · 내려간 것까지 찾기 · 내리기 · 복구
/users	등급 변경, 결제 내역 · 환불, 생년월일 정정
/worlds/*	세계관 목록 · 상세(숨긴 파트 목표) · 검수 · 이벤트 기록 · 운영 설정

04. API 연결

콘솔도 Nest만 부른다(FastAPI 직접 호출 금지). 처리는 app/actions/admin.ts 서버 액션이 Nest /admin/* 로 넘기고, 그림은 app/api/... 중계가 검수 대기 · 막힌 그림까지 no-store로 가져온다.

GET /admin/stats	대시보드 숫자
POST /admin/characters/:id/take-down	캐릭터 내리기(사유 필수)
POST /admin/content-reports/:id/block	그림 접근 차단(+계정 정지)
POST /admin/orders/:id/refund	전액 환불(결제사 승인 뒤 기록)
GET·PUT /admin/world-settings	하루 보상 횟수 · 이벤트 확률 · 간격

05. 내부 페이지



대시보드

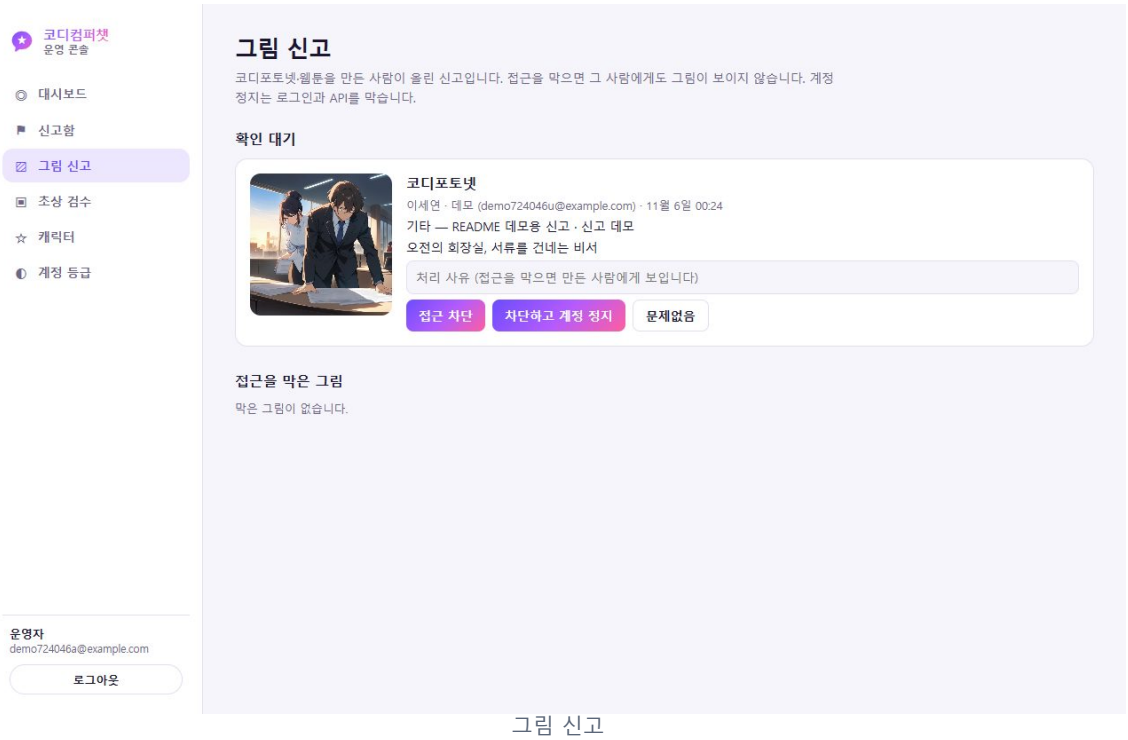


그림 신고

06. 디자인

이용자 웹과 같은 토큰을 쓰되 정보 밀도를 높였다. 숫자 줄 → 찾기 · 거르기 → 표 → 상세의 순서를 모든 화면에서 같게 해, 처음 보는 운영자도 같은 동작으로 일을 처리한다. 되돌릴 수 없는 처리(내리기 · 차단 · 환불)는 모두 사유를 받는다.

07. 문제점과 개선

운영 배포 연결	허용 IP 결정 전이라 운영 API에 연결하지 않았다(TASKS U08)
IP 거르기	Vercel에는 앞단 Caddy가 없어 앱 한 겹뿐 → 연결 전 헤더 신뢰 확인 필요
감사 로그	처리 기록은 DB에 남지만 별도 감사 화면이 없다

08. 미래 지향

- 운영자 처리 감사 로그 화면
- 신고 · 검수 큐의 알림(메일 · 웹훅)
- 역할 세분화(검수자 · 결제 담당)

03 · BACKEND

백엔드 서버 — 유일한 신뢰 경계

Nest.js 12(ESM) + Prisma 7 + PostgreSQL 16. 외부에 열린 유일한 백엔드로 인증 · 권한 · 소유 확인 · 요청 한도 · 코레딧 · 맥락 조립을 모두 맡는다. 판단 로직은 순수 함수로 모아 DB 없이 시험한다.

01. 개요

웹 · 콘솔은 Nest만 알고, Nest만 DB와 AI를 알고, AI는 DB를 모른다. 이 원칙 하나로 보안 검토와 시험 범위를 Nest 한 곳에 모았다. 모듈 30여 개, API 경로 155개, 테이블 37개.

- Controller → Guard(Auth · Admin · RateLimit) → Service → 순수 함수 / Prisma / ai-client / Credits
- 운영 기동 검사: NODE_ENV=production에서 REQUIRE_HTTPS · 토큰 길이 · 개발 스위치를 보고 어긋나면 시작하지 않는다
- 운영: Oracle Cloud A1.Flex(ARM) Docker Compose — Caddy · api · postgres · ai · ollama

02. 사용 엔진

런타임	Node.js 24 · Nest.js 12 (ESM)
ORM · DB	Prisma 7 · PostgreSQL 16 · 마이그레이션 34개
메일	Nodemailer (SMTP · 폴더 · 로그)
결제	토스페이먼츠 연동(보류) · 눌러서 프로(PRO_CLICK)
시험	Vitest 232개 — 순수 함수 *.spec.ts + 실제 PostgreSQL *.db.spec.ts

03. 상세 기능

character-chat	대화 한 턴: 맥락 조립 → /infer → 개인사 검사 → 저장
campaign-state	뒤에서 정리: 메시지 6개마다 요약 · 기억 · 친밀도 · 복선 · 퀘스트 판정
quests	비밀 퀘스트: 건넴 → 달성 → 보상 장면 한 번
worlds	월드코퍼챗: 세계관 · 플레이 · 3턴마다 파트 판정 · 미니게임 · 하루 보상 한도
studio	공방: 초안 → 3턴 테스트 → 발행(이메일 인증 계정만)
credits	코레딧: 출석 · 차감 · 실패 환불 · 보상을 DB 한 문장으로
persona-rag	2-gram BM25 + 키워드 보너스 지식 조각 검색(벡터 DB 없음)

04. API 연결

Nest 컨트롤러 경로 155개를 문서와 하나씩 대조해 빠진 것이 없음을 확인했다(2026-11-07). AI 호출은 대기열을 지나며, 화면에서 기다리는 답(foreground)이 뒤에서 하는 정리(background)보다 먼저 간다.

POST /auth/signup · /auth/login	가입 · 로그인 → 토큰(쿠키)
POST /characters/:id/chat	대화 한 턴 (답 · 사용량)
POST /worlds/:id/play/turn	월드 한 턴 (진행자 답 · 위치 · 이벤트)
POST /studio/characters/:id/test	발행 전 3턴 테스트
GET /health	상태 + AI 대기열(실행 · 대기 · 동시 수)

05. 내부 구조



시스템 구조



대화 한 턴의 처리 순서

06. 고급 기술

- 모델의 자기 확신을 믿지 않는다: 퀘스트 달성은 확신 ≥85, 인용 ≥8자, 인용이 그 화자의 실제 대사 안에 있을 때만 → 잘못 통과 0건
- 환각 억제: 설정에 없는 개인사를 묻는 턴만 /ground-check, 지어냈으면 다시 쓰기(최대 2번)
- AI 대기열: 동시 수 제한 + foreground 우선, 타임아웃 사슬(Vercel 300초 > API 280초 > 호스팅 45초)
- 돈과 횟수는 DB 한 문장(원자적 UPDATE)으로 세서 동시 요청에도 두 번 깎이지 않는다
- 웹과 API가 다른 기계일 때 프록시 토큰으로만 이용자 IP를 믿는다

07. 문제점과 개선

호스팅 백엔드의 모델 고르기	호스팅 API는 고른 모델 이름을 쓰지 않아 강화 무료 구분이 사라진다(T06)
성인 모드 가드	호스팅 API 서버에서 성인 모드를 코드로 막는 장치가 없다(T05)
메일	운영 SMTP 미정 → 인증 메일 미발송
백업	운영 DB 백업 cron 미설정(T04)

08. 추가할 백엔드 서버

- GPU 서버(Hetzner GEX45)로 그림 엔진 이전, ai · ollama만 분리 가능하게 설계됨
- 여러 대로 늘릴 때 요청 한도 저장소를 DB로(RATE_LIMIT_STORE=db)
- 결제 웹훅 수신 서버(토스)

09. 미래 지향

- SSE 스트리밍 응답
- 관측성: 요청 · AI 대기열 지표 대시보드
- 읽기 복제본과 백업 자동 복구 연습

04 · AI SERVICE

AI 서비스 — 연기 · 판정 · 그림

FastAPI(Python 3.12) 내부망 전용 서버. DB를 보지 않고 Nest가 조립해 넘긴 조각만 쓴다. 자체 가중치를 학습하지 않는 대신, 프롬프트 · 맥락 · 판정을 실제 모델로 재고 고쳤다.

01. 개요

연기(/infer), 요약 · 기억(/summarize · /remember), 환각 판정(/ground-check), 월드 진행자(/world/turn · /world/part-check), 그림(/photonet · /webtoon), 검수(/moderate-image) 등 경로 24개. 바깥에 열리지 않고 X-AI-Token이 맞는 요청만 받는다.

- LLM_BACKEND=openai: NVIDIA 호스팅 gemma-4-31b-it → 막히면 CPU Ollama gemma4:e4b
- LLM_BACKEND=ollama: 로컬 Ollama gemma4:e4b
- /train은 403 — v1은 학습하지 않는다는 정책

02. 사용 엔진

서버	FastAPI · Pydantic(camelCase 통신)
대화 모델	운영 NVIDIA 호스팅 gemma-4-31b-it, 로컬·예비 Ollama gemma4:e4b
그림	diffusers · Animagine XL 4.0 · SD 1.5 (NVIDIA GPU)
프리미엄	Anthropic Claude · xAI Grok (키가 있을 때만)
시험	unittest 71개 + 평가 스크립트 11묶음 + 퀘스트 판정 평가

03. 상세 기능

- 극소 카드 핵심 + 관련 지식 조각(최대 3개 · 700자)만 넣는 프롬프트 조립
- /ground-check: 답의 개인사가 설정 · 대화에 근거가 있는지 판정
- 월드 진행자: 여러 인물을 번갈아 연기하고 [그림: 이름표]·[이동: 이름 → 장소] 제어 줄을 낸다 → Nest가 검사 후 떼어 냄
- 코디포토넷: 줄거리를 읽고 장면을 먼저 설계(프롬프트) → 그림 → 검수
- 초상 · 장면 그림 검수: 성적 · 미성년자 · 잔혹 · 실존 인물 판정

04. 스크립트 · 평가 연결

학습 대신 평가로 고친다. services/ai/evals 의 스크립트가 실제 모델로 대화를 돌려 기준과 비교하고, 결과는 docs/trainings/ 에 날짜 · 표본과 함께 남긴다.

말투 유지	12/12 (기준 90%)
"너 AI지?" 유도	12/12 (95%)
개인사 단정하지 않기	11/12 (90%)
장면 전환 나레이션	7/12 → 48/48
퀘스트 보상 장면	2/8 → 20/20
퀘스트 판정 잘못 통과	0건
월드 진행자 형식	23/24 (80%)

05. 내부 화면



월드 한 턴의 흐름



웹툰 콘티와 컷

06. 고급 기술

- 판단은 코드로: 작은 모델에게 조건부 지시 대신 '이번 답 첫 줄은 @:' 같은 할 일 하나만 → 나레이션 7/12 → 48/48
- 보상 장면은 코드가 갈래를 나누고 모델은 번호만 고른다 → 2/8 → 20/20
- 예비 모델 자동 전환: 429 · 5xx · 시간 초과면 같은 요청을 CPU Ollama로
- 이 포트폴리오의 챗봇 코드: LangChain + LangGraph + BM25 RAG(Flask). 공개 사이트는 이 서버를 올리지 않아 모델을 부르지 않는다

07. 문제점과 개선

31B 품질 미측정	평가는 gemma4:e4b로 맞춰 둠 → 호스팅 31B 재평가 필요(Q08)
데이터 반출	호스팅 API로 대화 · 검수 입력이 나감 → 처리방침 반영 필요(U11)
웹툰 일관성	컷 사이 옷 색이 달라진다(Q04)
장소 이동	월드 진행자가 이동 제어를 거의 쓰지 않는다(0/24)

08. 추가할 서버

- GPU 그림 노드(deploy/ai-node) 분리 운영
- 평가 자동화를 CI의 야간 작업으로
- 임베딩 기반 하이브리드 검색(BM25 + 벡터) 실험

05 · FUTURE

미래 지향

공개 서비스로 가기 위해 남은 일과, 그다음에 하고 싶은 일을 순서대로 정리했다.

01. 공개 전 반드시

- 처리방침 국외 이전 표를 실제 구성(NVIDIA · Vercel · Oracle)에 맞추고 법률 검토
- 운영 메일(SMTP) 연결 → 인증 메일
- 운영 콘솔 연결(허용 IP)
- 호스팅 31B 모델로 평가 재실행
- 저장소 비공개 전환

02. 제품

- 스트리밍 응답
- GPU 서버로 코디포토넷 · 웹툰 공개
- 월드 진행자의 장소 이동 · 긴 플레이 요약
- 세계관 신고 화면

03. 기술

- 하이브리드 검색(BM25 + 임베딩)
- 평가 자동화와 회귀 대시보드
- 관측성(지표 · 추적)
- 여러 대 운영(요청 한도 DB 저장소, AI 노드 분리)